



Proactive Scaling of UPF in 5G Networks Based on Predicted Traffic with the use of LSTMs

Gabriele Argentieri: 21502063

Isaac Balam Rodriguez Arellano: 21412131

Vealanganny Angele Irudayanathan: 21304987

Academic Year 2025–2026

Date: 12 January 2026

1 Introduction

The objective of this project is to design and validate an autoscaling controller for a 5G Core User Plane Function (UPF) that performs *proactive* scaling actions based on short-horizon throughput prediction. The primary reliability goal is to prevent sustained operation above the physical throughput capacity of a single UPF instance (assumed to be 15 Mbps in this report).

Proactive scaling differs from reactive scaling in the signal that triggers the control action. A reactive controller typically observes a threshold violation on a measured metric (e.g., throughput or CPU) and then attempts to compensate *after* the system has already entered a high-load regime. In Kubernetes environments, however, the control loop includes non-trivial actuation delay due to pod scheduling and initialization, as well as additional application-level warm-up. Consequently, reacting only after a threshold breach may lead to transient congestion, packet loss, or QoS degradation during the time required to instantiate the additional UPF.

The proposed design instead predicts the next-step throughput using an LSTM model and triggers Scale-Out based on the forecasted value. To compensate for model prediction lag (inertia) and Kubernetes actuation latency, the controller uses a *safety threshold* of 12 Mbps, which is intentionally lower than the physical limit of 15 Mbps. This margin biases decisions toward earlier Scale-Out, prioritizing service continuity over minimal resource usage.

2 5G Network Deployment Baseline

The experimental environment utilizes the OpenAirInterface (OAI) 5G Core, orchestrated within a Kubernetes cluster. The fundamental control-plane infrastructure and database components were instantiated following the reference procedure provided. Consequently, this report focuses on the high-level architectural integration rather than the standard deployment steps.

The baseline configuration is managed through Helm charts, ensuring proper sequencing of network function dependencies. While the primary User Plane Function (UPF1) and the initial UE emulator (UE1) constitute the static substrate of the network, the core contribution of this project lies in the *on-demand* elasticity. Specifically, the autoscaler dynamically manages the lifecycle of a secondary UPF (UPF2) and an additional traffic generator (UE2). This architectural split allows the system to treat the 5G Core as a stable foundation while testing the proactive scaling capabilities on the volatile user-plane components.

3 Autoscaler Design

3.1 Control-loop structure and state

The autoscaler operates through a periodic loop that processes throughput data from a CSV file at one-second intervals. The use of a CSV trace acts as a deterministic driver for reproducibility and debugging purposes, serving as a proxy for real-time telemetry metrics (e.g., Prometheus) that would be observed in a production environment. At

each time step t , the controller maintains a fixed-size history buffer of the most recent N throughput samples, where $N = 20$ corresponds to the LSTM lookback window. The controller computes a one-step-ahead forecast $\hat{y}_{t+1}$ and applies a threshold decision policy:

- **Scale-Out condition:** if $\hat{y}_{t+1} > 12Mbps$ and the system is currently operating with a single UPF, the controller deploys a second UPF (UPF2) and its associated UE generator.
- **Scale-In condition:** if $\hat{y}_{t+1} \leq 12Mbps$ and the system is currently scaled out, the controller removes UPF2 and the second UE.

3.2 Safety threshold selection

The decision threshold is set to 12 Mbps, below the single-UPF capacity of 15 Mbps. This choice addresses two latency sources:

- **Inference dynamics:** LSTM predictions, especially under rapid changes, can exhibit lag relative to the true trajectory. Using a lower trigger value reduces the probability that a forecast crosses the threshold only after the underlying process has already approached saturation.
- **Actuation latency:** Kubernetes Scale-Out is not instantaneous. Helm deployment, pod scheduling, image pull, and readiness conditions introduce a delay between decision and effective capacity increase. Triggering earlier improves the likelihood that UPF2 becomes ready before the offered load exceeds the single-UPF capacity.

This ensures that the second UPF is active and ready before the real traffic reaches the breaking point.

3.3 Simulation timing and determinism

To enable fast iteration, the experiment compresses time steps to 1 s per sample. Each step is synchronized with traffic generation by running `iperf` for a duration of 1 s. The controller uses blocking I/O for traffic generation (i.e., the loop waits until traffic generation completes before advancing). This blocking behavior enforces temporal determinism: each predicted decision corresponds to a well-defined interval of traffic injection.

A 20-step warm-up phase is required due to the lookback window. During this initial “blind” period, the predictive model cannot produce valid forecasts because the input vector is incomplete. The implementation handles this by initializing the history buffer with the first 20 samples from the trace and beginning inference at step 20. This choice avoids artificial padding and maintains semantic consistency: forecasts are based solely on observed values.

3.4 Helm/Kubernetes integration

Scale-Out and Scale-In operations are implemented via Helm and `kubectl` invocations. The Scale-Out routine deploys the UPF2 Helm chart and applies a Kubernetes manifest for the second UE generator. The controller *blocks* execution until the dependency chain (UE2 and UPF2) reports a **Ready** status. This mandatory waiting time constitutes the actuation latency that necessitated the use of the safety threshold defined in Section 3.2.

Conversely, Scale-In performs the reverse operations (uninstalling the Helm release and deleting the UE2 manifest) to release resources.

4 Predictive Engine

4.1 Model Training and Artifact Generation

The LSTM model is trained offline using a supervised learning approach. The process, implemented in `train.py`, is designed to capture the temporal dependencies of the throughput signal through several key steps:

- **Data Preprocessing:** The dataset is sorted by timestamp and split into training (90%) and testing (10%) sets. A `MinMaxScaler` is fitted exclusively on the training data to prevent data leakage and ensure unbiased evaluation.
- **Sliding Window Transformation:** The raw time series is converted into a supervised learning format using a sliding window approach. With a lookback of $N = 20$ and a horizon of $H = 1$, the model learns to map the sequence $\{y_{t-19}, \dots, y_t\}$ to the subsequent target y_{t+1} .
- **Model Optimization:** The model uses Mean Squared Error (MSE) as loss and incorporates Early Stopping to prevent overfitting.
- **Artifact Storage:** Upon completion, the trained model (`.keras`), the scaler (`.pkl`), and a metadata file (`.json`) are exported to the `artifacts/` directory, creating a versioned package for the deployment phase.

4.2 Inference Pipeline and Runtime Execution

Once deployed, the predictive engine transitions to an operational mode where it processes live throughput data to generate real-time forecasts. The execution begins with the retrieval of the pre-trained artifacts from the storage directory. To optimize performance and maintain a stable one-second control loop, the system implements a caching strategy that avoids the high overhead of reloading the model at each iteration.

The LSTM model processes the input sequence to produce a normalized one-step-ahead prediction, which is immediately transformed back to its original scale in Mbps. This final value is then passed to the decision logic, providing the necessary lead time to trigger scaling actions before congestion occurs.

5 Experimental Evaluation

5.1 Simulation Environment and Data Traces

Validation is performed using a trace-driven simulation approach: a CSV file named `test_dataset.csv` contains a timestamped sequence of throughput targets. Each value is interpreted as the offered load to be generated via `iperf` in the corresponding time step.

This strategy is chosen to ensure repeatability and controlled experimentation. By using a fixed trace, scaling decisions can be correlated with known future traffic transitions,

and changes to controller parameters (e.g. threshold) can be evaluated under identical conditions.

5.2 Traffic design

The dataset is intentionally constructed with linear ramps rather than step functions. Sudden jumps (e.g. 8 Mbps to 28 Mbps in one step) are difficult for sequence models to anticipate from short history windows; they can induce prediction lag and, consequently, late scaling decisions.

Ramp transitions (e.g. 8 Mbps $\rightarrow$ 9.5 Mbps $\rightarrow$ 11 Mbps $\rightarrow \dots$) provide a consistent growth trend that an LSTM can learn to extrapolate. This allows the evaluation to focus on the intended proactive behavior: scaling based on the inferred trajectory rather than reacting to an instantaneous discontinuity.

The use of a peak value of 28 Mbps serves as a stress-test for the combined infrastructure. Since this value resides at $\approx 186\%$ of a single UPF's capacity, it creates a deterministic requirement for the second UPF to be fully operational and ready to route packets. The transition from 8 Mbps (steady state) to 28 Mbps via linear ramps allows for the observation of the controller's responsiveness to varying slopes, simulating a realistic surge in UE attachment or high-bandwidth service demand.

5.3 Stress testing Scale-Out and Scale-In

The trace includes a peak of approximately 28 Mbps. This peak is selected to exceed the single-UPF capacity of 15 Mbps while remaining within the combined capacity of two UPFs (nominally 30 Mbps). As a result, the scenario stresses both:

- **Scale-Out correctness:** the controller must deploy UPF2 before the offered load would saturate UPF1.
- **Scale-In correctness:** when the ramp descends, the controller should eventually remove UPF2 after the predicted load stabilizes below the safety threshold.

5.4 Analysis of Experimental Results

The proactive behavior of the system is empirically validated by the synchronization between the predicted trend and the scaling status. As shown in the simulation results, the transition to Dual UE is consistently triggered while the actual throughput is still within the 12 Mbps to 14 Mbps range.

This early intervention is critical: although the LSTM model slightly underestimates the absolute peak (stabilizing at approximately 26 Mbps against a target of 28 Mbps), the forecast successfully remains above the safety threshold. This behavior confirms that the controller prioritizes *state-switching reliability* over exact value regression. Furthermore, the Scale-In phase exhibits an implicit stabilizing effect similar to hysteresis; the gradual descent of the predicted curve prevents premature de-allocation of resources while the real traffic is still volatile, effectively acting as a low-pass filter for the scaling logic.

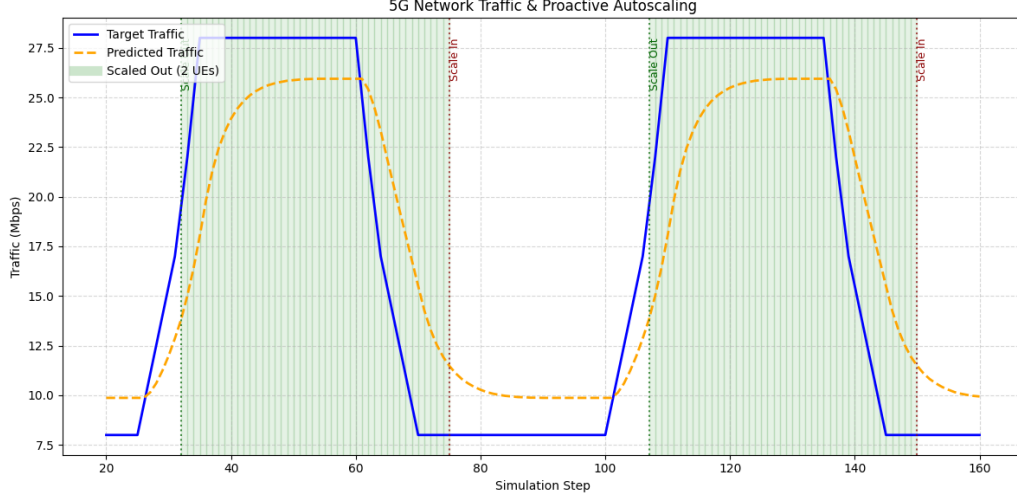


Figure 1: The orange dashed line represents the LSTM prediction crossing the safety threshold to trigger the Scale-Out phase (green shaded area) before physical saturation occurs.

6 Conclusion

This project demonstrates a practical architecture for proactive UPF scaling that explicitly accounts for actuation latency and prediction dynamics. The autoscaler combines a trace-driven simulation framework with a LSTM forecast and a conservative safety threshold to trigger Scale-Out before a single instance gets saturated.

From a systems perspective, the primary engineering insight is that end-to-end reliability depends not only on predictive accuracy but also on timing: inference overhead, controller sampling period, and Kubernetes readiness delays jointly determine whether scaling actions are effective. The adopted design choices collectively provide a coherent implementation aligned with network performance and stability requirements.